

# 工程项目协同管理系统 - 交付文档

版本 v2.0 Enterprise

技术栈 React + PocketBase

状态 已交付

版本: v2.0 Enterprise

交付日期: 2026-02-04

技术栈: PocketBase + React + TypeScript + Vite + Capacitor

适用平台: Web (PC/Mobile) + Android APK

## 文档概述

本文档是工程项目协同管理系统的完整交付文档，包含：

- 产品设计**：PRD、用户角色、功能模块
- 技术架构**：系统架构、技术栈、目录结构
- 数据设计**：数据库 Schema、字段定义、访问规则
- 界面设计**：设计规范、页面截图
- 部署指南**：服务器部署、数据库初始化
- 使用手册**：操作流程、常见问题
- API 文档**：接口说明、代码示例
- 安全设计**：认证、权限、数据安全
- 维护手册**：备份、日志、脚本工具

## 目录

- [1. 产品概述](#)
- [2. 核心功能](#)
- [3. 技术架构](#)
- [4. 数据库设计](#)
- [5. 界面设计](#)
- [6. 部署指南](#)
- [7. 使用指南](#)
- [8. API 接口](#)
- [9. 安全设计](#)
- [10. 维护手册](#)

## 1. 产品概述

## 1.1 产品定位

本产品是一款专为工程管理公司打造的全流程协同管理平台。通过"三步法"（标准化拆解、可视化追踪、流程化配合）解决多项目并行下的信息断层与协作内耗问题。

## 1.2 目标用户

| 角色       | 核心诉求      | 主要功能                 |
|----------|-----------|----------------------|
| 老板/决策层   | 全局掌控、风险预警 | AI 简报、风险看板、异常任务聚焦    |
| 项目经理     | 流程管控、团队协调 | 项目看板、审核中心、任务编排、AI 问答 |
| 执行层（资料员） | 明确任务、高效执行 | 我的任务、卡点上报、交接提案       |

## 1.3 核心价值

- 标准化**：将复杂工程拆解为标准化任务节点
- 可视化**：看板+甘特图，一眼看清进度
- 闭环化**：强制"完成即交接"，杜绝推诿

## 2. 核心功能

### 2.1 功能模块总览





## 2.2 功能详细说明

### 2.2.1 认证模块

| 功能    | 说明             | 实现状态 |
|-------|----------------|------|
| 用户名登录 | 支持用户名 + 密码登录   | ✓    |
| 邮箱登录  | 支持邮箱 + 密码登录    | ✓    |
| 记住登录  | 勾选后 Token 长期有效 | ✓    |
| 自动登录  | 刷新页面自动恢复会话     | ✓    |
| 用户注册  | 填写信息注册新用户      | ✓    |
| 退出登录  | 清除 Token 并跳转   | ✓    |
| 密码修改  | 在设置页修改密码       | ✓    |

登录流程代码示例：

```
// src/pages/Login.tsx
const handleLogin = async () => {
  try {
    const authData = await pb.collection('users')
      .authWithPassword(username, password)

    Toast.show({ icon: 'success', content: '登录成功' })

    // 根据角色跳转
    if (authData.record.role === 'admin' || authData.record.role === 'manager') {
      navigate('/admin')
    } else {
      navigate('/app/tasks')
    }
  } catch (error) {
    Toast.show({ icon: 'fail', content: '登录失败' })
  }
}
```

## 2.3 项目看板 (Kanban)

功能特性:

| 特性   | 描述                     |
|------|------------------------|
| 列定义  | 待开始、进行中、卡点、已逾期、已完成     |
| 卡片信息 | 任务名、负责人头像、截止日期、优先级标签   |
| 拖拽交互 | 移动端长按 250ms / PC 端直接拖动 |
| 即时反馈 | 乐观更新，网络失败自动回滚          |
| 列内排序 | 支持拖拽调整任务顺序             |
| 跨列拖拽 | 支持拖拽到不同状态列             |

技术实现:

```
// src/components/kanban/kanbanBoard.tsx
import { DndContext, closestCorners, DragOverlay } from '@dnd-kit/core'
import { SortableContext, verticalListSortingStrategy } from '@dnd-kit/sortable'

// 拖拽结束处理
const handleDragEnd = (event: DragEndEvent) => {
  const { active, over } = event

  if (!over) return

  const activeId = active.id as string
  const overId = over.id as string

  // 找到任务和目标列
  const activeTask = tasks.find(t => t.id === activeId)
  const overColumn = columns.find(c => c.id === overId)

  if (activeTask && overColumn) {
    // 乐观更新
    const newStatus = overColumn.id as TaskStatus
    updateTask.mutate({
      id: activeId,
      data: { status: newStatus }
    })
  }
}
```

状态颜色映射:

```
/* 待开始 */
--status-pending: #64748B;
/* 进行中 */
--status-in-progress: #3B82F6;
/* 卡点 */
--status-blocked: #F59E0B;
/* 已逾期 */
--status-overdue: #EF4444;
/* 已完成 */
--status-completed: #10B981;
```

## 2.4 项目时间轴（甘特图）

功能特性：

| 特性    | 描述             |
|-------|----------------|
| 人员分组  | 任务按负责人分行显示     |
| 时间刻度  | 显示日期标尺         |
| 缩放范围  | 50% - 200% 缩放  |
| 移动端缩放 | 双指捏合缩放         |
| 今日标记  | 红色虚线标记当前日期     |
| 任务条   | 显示任务名称、状态颜色、进度 |
| 详情查看  | 点击任务条弹出详情抽屉    |

技术实现：

```
// src/pages/ProjectTimeline.tsx

// 计算任务条位置
const calculateTaskPosition = (task: Task, timelineStart: Date, daywidth: number) => {
  const startDate = new Date(task.start_date || task.created)
  const endDate = new Date(task.deadline || task.created)

  const startOffset = Math.floor(
    (startDate.getTime() - timelineStart.getTime()) / (1000 * 60 * 60 * 24)
  )
  const duration = Math.ceil(
    (endDate.getTime() - startDate.getTime()) / (1000 * 60 * 60 * 24)
  ) + 1

  return {
    left: startOffset * daywidth,
    width: duration * daywidth,
```

```

    }
  }

  // 缩放处理
  const handleZoom = (delta: number) => {
    setZoom(prev => Math.min(200, Math.max(50, prev + delta)))
  }

```

甘特图样式：

```

.timeline-task-bar {
  height: 28px;
  border-radius: 4px;
  display: flex;
  align-items: center;
  padding: 0 8px;
  font-size: 12px;
  color: white;
  cursor: pointer;
  transition: transform 0.2s;
}

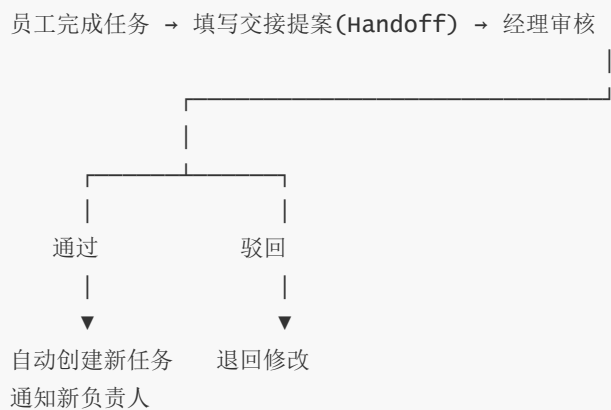
.timeline-task-bar:hover {
  transform: scaleY(1.1);
}

.timeline-today-line {
  position: absolute;
  width: 2px;
  height: 100%;
  background: #EF4444;
  border: 1px dashed #EF4444;
  z-index: 10;
}

```

## 2.4 交接闭环

流程设计：



### 业务规则：

- 员工不能直接创建任务，必须通过交接提案
- 经理审核通过后系统自动创建下一步任务
- 所有操作记录到 `audit_logs`

### 代码实现 (`api.ts`)：

```
// 审核通过交接
export function useApproveHandoff() {
  return useMutation({
    mutationFn: async ({ handoffId, reviewNote }) => {
      const handoff = await pb.collection('handoffs').getOne(handoffId)

      // 1. 创建新任务
      const newTask = await pb.collection('tasks').create({
        project: handoff.project,
        stage_name: handoff.proposed_title,
        assignees: handoff.proposed_assignees,
        deadline: handoff.proposed_due_date,
        status: 'pending',
      })

      // 2. 更新交接状态
      await pb.collection('handoffs').update(handoffId, {
        status: 'approved',
        reviewer: pb.authStore.model.id,
        review_note: reviewNote,
        approved_task: newTask.id,
      })

      // 3. 发送通知给新负责人
      for (const userId of handoff.proposed_assignees) {
        await pb.collection('notifications').create({
          user: userId,
          type: 'task_assigned',
          title: '新任务分配',
          content: `您被分配了新任务: ${handoff.proposed_title}`,
        })
      }

      // 4. 记录审计日志
      await pb.collection('audit_logs').create({
        action_type: 'approve_handoff',
        operator: pb.authStore.model.id,
        note: `审核通过: ${handoff.proposed_title}`,
      })
    }
  })
}
```

## 2.5 卡点上报

卡点信息结构：

```
interface Blocker {
  reason_type: 'waiting_approval' | 'waiting_materials' | 'technical_issue' |
  'other'
  reason_detail: string           // 详细描述
  need_help_from: string[]        // 需要谁协助
  expected_resolve: string        // 期望解决时间
  reported_at: string             // 上报时间
}
```

前端交互：

- 点击"上报卡点"按钮
- 弹出表单填写卡点信息
- 提交后任务状态变为 blocked
- 自动通知相关经理

## 2.6 AI 智能助手

功能特性：

| 功能     | 描述                   | 权限       |
|--------|----------------------|----------|
| 实时诊断报告 | 聚合项目数据，AI 生成风险分析     | manager+ |
| 智能问答   | 基于项目上下文的对话           | manager+ |
| 风险等级   | 自动评估 low/medium/high | 自动       |
| 简报历史   | 保存历史简报供查阅            | manager+ |
| 模型选择   | 支持切换 AI 模型           | admin    |

AI 服务配置：

| 配置项    | 值                                              |
|--------|------------------------------------------------|
| AI 提供商 | SiliconFlow                                    |
| 默认模型   | DeepSeek-V3                                    |
| API 端点 | https://api.siliconflow.cn/v1/chat/completions |
| 响应格式   | JSON（简报） / 文本流（问答）                             |
| 超时时间   | 30 秒                                           |
| Key 存储 | localStorage('sf_api_key')                     |



## AI 简报生成代码：

```
// src/lib/ai-service.ts

export async function generateAIReport(projectData: ProjectData): Promise<AIReport>
{
  const apiKey = localStorage.getItem('sf_api_key')
  const model = localStorage.getItem('ai_model') || 'deepseek-ai/DeepSeek-V3'

  const systemPrompt = `你是一位专业的工程项目管理顾问。请基于以下项目数据，生成一份简洁的风险诊断报告。

  输出格式（JSON）：
  {
    "overall_assessment": "整体评估（1-2句话）",
    "progress_summary": "进度摘要",
    "blocked_analysis": "卡点分析（如有）",
    "risk_warnings": ["风险预警1", "风险预警2"],
    "recommendations": ["建议1", "建议2"],
    "risk_level": "low|medium|high"
  }`

  const response = await fetch('https://api.siliconflow.cn/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${apiKey}`,
    },
    body: JSON.stringify({
      model,
      messages: [
        { role: 'system', content: systemPrompt },
        { role: 'user', content: JSON.stringify(projectData) },
      ],
      response_format: { type: 'json_object' },
    }),
  })

  const data = await response.json()
  return JSON.parse(data.choices[0].message.content)
}
```

## AI 问答示例：

用户：当前最大的风险是什么？

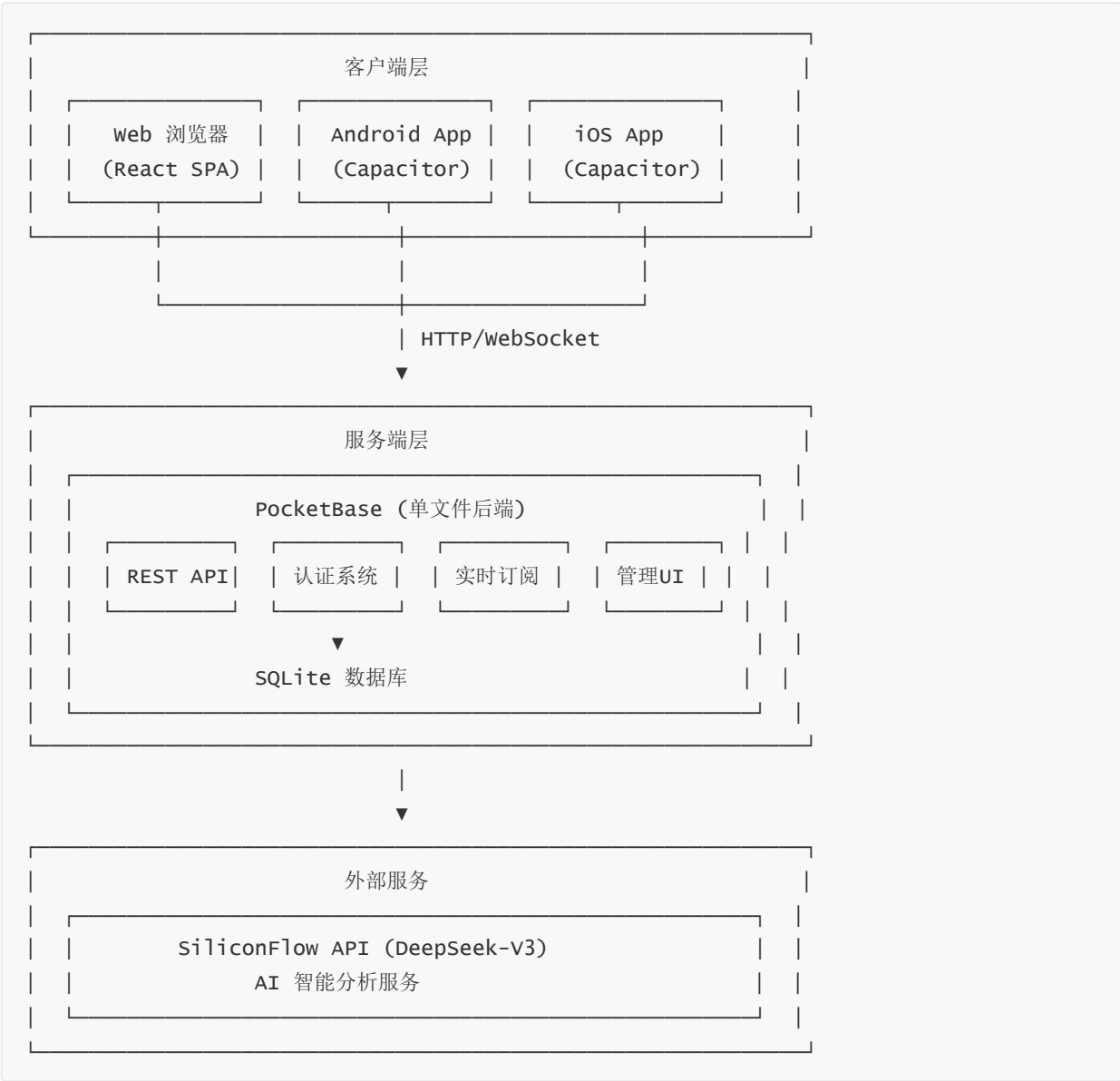
AI：根据当前项目数据分析，最大风险是：

- 1. **\*\*材料供应延迟\*\***：凤凰山跨海大桥工程有 2 个任务因等待材料处于卡点状态
- 2. **\*\*审批流程缓慢\*\***：港口物流园区项目有 1 个任务等待甲方审批超过 5 天
- 3. **\*\*人员负载不均\*\***：张工程师当前负责 8 个任务，可能影响进度

建议优先协调材料供应商，并与甲方沟通加快审批流程。

### 3. 技术架构

#### 3.1 系统架构图



### 3.2 技术栈详情

| 层级    | 技术                | 版本    | 说明          |
|-------|-------------------|-------|-------------|
| 前端框架  | React             | 19.x  | UI 构建       |
| 类型系统  | TypeScript        | 5.9+  | 类型安全        |
| 构建工具  | Vite              | 6.0+  | 开发&打包       |
| UI 组件 | Ant Design Mobile | 5.42+ | 移动端组件       |
| 状态管理  | TanStack Query    | 5.x   | 服务端数据       |
| 状态管理  | Zustand           | 5.x   | UI 状态       |
| 动画    | Framer Motion     | 12.x  | 微交互动效       |
| 图表    | ECharts           | 5.x   | 数据可视化       |
| 后端    | PocketBase        | 0.26+ | BaaS        |
| 数据库   | SQLite            | -     | 嵌入式数据库      |
| 移动打包  | Capacitor         | 5.x   | Android/iOS |
| AI 服务 | SiliconFlow       | -     | DeepSeek-V3 |

### 3.3 前端目录结构

```
v2/frontend/src/
├─ App.tsx           # 路由配置
├─ main.tsx          # 应用入口
├─ index.css         # 全局样式
├─
├─ components/       # 可复用组件
│   └─ kanban/        # 看板组件
│       └─ KanbanBoard.tsx # 看板主容器
│       └─ KanbanColumn.tsx # 看板列
│       └─ TaskCard.tsx   # 任务卡片
│       └─ TaskDetailDrawer.tsx
│   └─ dashboard/
│       └─ AISummaryCard.tsx # AI 简报卡片
│   └─ EmptyState.tsx      # 空状态
│   └─ Skeleton.tsx       # 骨架屏
├─
├─ lib/              # 核心库
│   └─ pocketbase.ts      # PB 客户端
│   └─ api.ts             # API Hooks
│   └─ ai-service.ts      # AI 服务
│   └─ queryClient.ts     # Query 配置
│   └─ store.ts           # Zustand Store
```

```
|
└─ pages/                                # 页面组件
    └─ admin/
        │ └─ AdminDashboard.tsx # 管理后台
        │ └─ AIConsole.tsx      # AI 控制台
        └─ Login.tsx            # 登录
    └─ Register.tsx             # 注册
    └─ Home.tsx                 # 首页
    └─ ProjectTimeline.tsx      # 时间轴
    └─ ProjectKanban.tsx        # 看板
    └─ TaskDetail.tsx           # 任务详情
    └─ MyTasks.tsx              # 我的任务
    └─ ManagerDashboard.tsx     # 经理工作台
    └─ ReviewCenter.tsx         # 审核中心
    └─ Profile.tsx              # 个人中心
    └─ SettingsPage.tsx         # 设置
    └─ Notifications.tsx       # 通知
```

## 4. 数据库设计

### 4.1 集合概览

| 集合名           | 类型   | 说明    | 记录数示例 |
|---------------|------|-------|-------|
| users         | Auth | 用户表   | 21    |
| projects      | Base | 项目表   | 5     |
| tasks         | Base | 任务表   | 79    |
| handoffs      | Base | 交接提案  | 3     |
| comments      | Base | 评论    | 10    |
| audit_logs    | Base | 操作日志  | 11    |
| notifications | Base | 通知    | 15    |
| ai_summaries  | Base | AI 简报 | 2     |

### 4.2 核心字段定义

users (用户)

```
{
  id: "RECORD_ID",
  username: "zhang_manager",
  email: "zhang@example.com",
  name: "张经理",
  avatar: "avatar.jpg",
  role: "manager",          // admin | manager | employee
  department: "工程部",
  position: "项目经理",
  flower_count: 15          // 红花数（可选）
}
```

## projects (项目)

```
{
  id: "PROJECT_ID",
  name: "凤凰山跨海大桥工程",
  code: "ENG-2026-001",
  status: "active",         // active | completed | archived
  description: "大型桥梁建设项目",
  manager: "USER_ID",
  start_date: "2026-01-01",
  end_date: "2026-12-31",
  progress: 45,
  total_tasks: 22,
  completed_tasks: 10
}
```

## tasks (任务)

```
{
  id: "TASK_ID",
  project: "PROJECT_ID",
  stage_name: "地基验收",
  description: "验收详情...",
  status: "in_progress",    // pending | in_progress | blocked | completed | overdue
  assignees: ["USER_ID_1", "USER_ID_2"],
  created_by: "USER_ID",
  start_date: "2026-02-01",
  deadline: "2026-02-05",
  sequence: 1000,           // 看板排序
  priority: "high",         // low | normal | high
  is_milestone: true,
  blocker: {                // 卡点信息 (JSON)
    reason_type: "waiting_approval",
    reason_detail: "甲方未确认图纸",
    need_help_from: ["USER_ID"],
    reported_at: "2026-02-03T10:00:00Z"
  },
  completed_steps: "1. 已完成...",
}
```

```
next_steps: "1. 下一步..."
}
```

handoffs（交接提案）

```
{
  id: "HANDOFF_ID",
  project: "PROJECT_ID",
  from_task: "TASK_ID",
  proposed_title: "主体施工",
  proposed_description: "进入主体结构施工阶段",
  proposed_assignees: ["USER_ID"],
  proposed_due_date: "2026-03-01",
  status: "pending",           // pending | approved | rejected
  submitter: "USER_ID",
  reviewer: "USER_ID",
  review_note: "同意，注意安全",
  approved_task: "TASK_ID"    // 审核通过后创建的新任务
}
```

4.3 状态枚举

任务状态

| 值           | 含义  | 颜色         |
|-------------|-----|------------|
| pending     | 待开始 | 灰色 #64748B |
| in_progress | 进行中 | 蓝色 #3B82F6 |
| blocked     | 卡点  | 橙色 #F59E0B |
| completed   | 已完成 | 绿色 #10B981 |
| overdue     | 已逾期 | 红色 #EF4444 |

用户角色

| 值        | 含义   | 权限         |
|----------|------|------------|
| admin    | 管理员  | 全部权限       |
| manager  | 项目经理 | 项目管理、审核、AI |
| employee | 员工   | 任务执行、卡点上报  |

## 4.4 访问规则

| 集合         | List | View | Create | Update  | Delete |
|------------|------|------|--------|---------|--------|
| users      | 登录用户 | 登录用户 | 开放注册   | 仅本人     | 管理员    |
| projects   | 登录用户 | 登录用户 | 经理+    | 经理+     | 管理员    |
| tasks      | 登录用户 | 登录用户 | 经理+    | 经理+或负责人 | 管理员    |
| handoffs   | 登录用户 | 登录用户 | 登录用户   | 经理+     | 管理员    |
| audit_logs | 登录用户 | 登录用户 | 登录用户   | 禁止      | 管理员    |

## 5. 界面设计

### 5.1 设计规范

#### 色彩系统

```
/* 品牌色 */
--primary-color: #3B82F6;      /* 主色调: 蓝色 */
--accent-color: #3B82F6;       /* 强调色 */

/* 语义色 */
--success-color: #10B981;      /* 成功: 绿色 */
--warning-color: #F59E0B;      /* 警告: 橙色 */
--danger-color: #EF4444;       /* 危险: 红色 */
--neutral-color: #64748B;      /* 中性: 灰色 */

/* 背景色 */
--background-color: #F8FAFC;   /* 页面背景 */
--card-bg: #FFFFFF;            /* 卡片背景 */
```

#### 阴影系统

```
/* 轻微阴影 */
--shadow-sm: 0 1px 2px rgba(0,0,0,0.05);

/* 中等阴影 */
--shadow-md: 0 4px 6px rgba(0,0,0,0.1);

/* 高级阴影（玻璃态） */
--shadow-premium: 0 8px 32px rgba(0,0,0,0.1);
```

## 5.2 页面截图

截图保存在 `v2/docs/screenshots/` 目录

可运行 `python v2/scripts/screenshot_pages.py` 重新生成

---

### 5.2.1 移动端界面

#### 登录页面

极简设计，支持用户名/邮箱登录，品牌标识清晰。





# 工程结算管理

Enterprise Project Management



服务器已连接



用户名 / 邮箱



密码



记住登录状态

登录

## 测试账号

经理: wang\_eng / 12345678

员工: li\_audit / 12345678

**还没有账号? 立即注册 →**

### 工作进展 (任务列表)

按项目分组展示任务，直观的项目卡片设计。



## 工作进展

项目与进度

轨道交通5号线站点配套

🕒 进行中

当前进度

0%

市中心老旧小区改造三期

🕒 进行中

当前进度

0%

# AA智慧产业园弱电项目

🕒 进行中



当前进度

0%



工作进展



我的

## 个人中心

用户信息、统计数据、快捷入口、头像管理。



## 工作进展

项目与进度

轨道交通5号线站点配套

🕒 进行中

当前进度

0%

市中心老旧小区改造三期

🕒 进行中

当前进度

0%

# AA智慧产业园弱电项目

🕒 进行中



当前进度

0%



工作进展



我的

## 经理工作台

关键指标卡片、待办事项、快速操作入口。



## AI 智能决策简报

全局监控 · 01/31 03:56

高风险

### 🚨 全局风险预警

当前风险等级: ● 高危

主要原因: 2个核心项目存在关键路径阻塞, 团队整体逾期率上升15%。

### 👥 团队效能分析 (Team Performance)

- ⚠️ 需关注人员 (Abnormal):
  - 李工: 负责的"XX道路"项目有 **3项** 任务严重逾期, 且连续2周末更新进度。
  - 张工: 配合度较低, 多个任务卡在"待提交"状态超过5天。
- 🌟 表现优异 (Excellent):
  - 王经理: 本周提前完成 **5项** 关键节点任务, 并在"防汛应急"项目中表现突出。
  - 刘工: 零逾期, 交付质量评分 9.8/10。

### 📉 核心卡点 (Critical Issues)

- XX道路结算项目:** 审计局反馈隐蔽资料缺失, 已停滞4天。

2. 桥梁工程: 钢筋材料进场受阻, 导致后续浇筑任务延期。

✂ 建议执行方案

- **立即介入:** 请项目经理直接对接审计局, 明确缺失资料清单。
- **人员辅导:** 安排资深工程师辅导李工, 梳理逾期任务优先级。
- **激励机制:** 在周会上公开表扬王经理和刘工, 树立标杆。

\*(Generated by DeepSeek-V3.2 Enterprise Mode)\*



更新数据



向 AI 提问

Powered by DeepSeek-V3.2



12

进行中项目

0

待审核交接

32

逾期任务

0

卡点任务



 进入审核中心

 异常任务

图纸审核

XX道路结算项目

逾期



材料送检

YY桥梁结算项目

逾期



竣工验收

XX大道市政改造工程

逾期



主体施工

YY商业中心主体建设

逾期



前期准备

ZZ河道治理二期

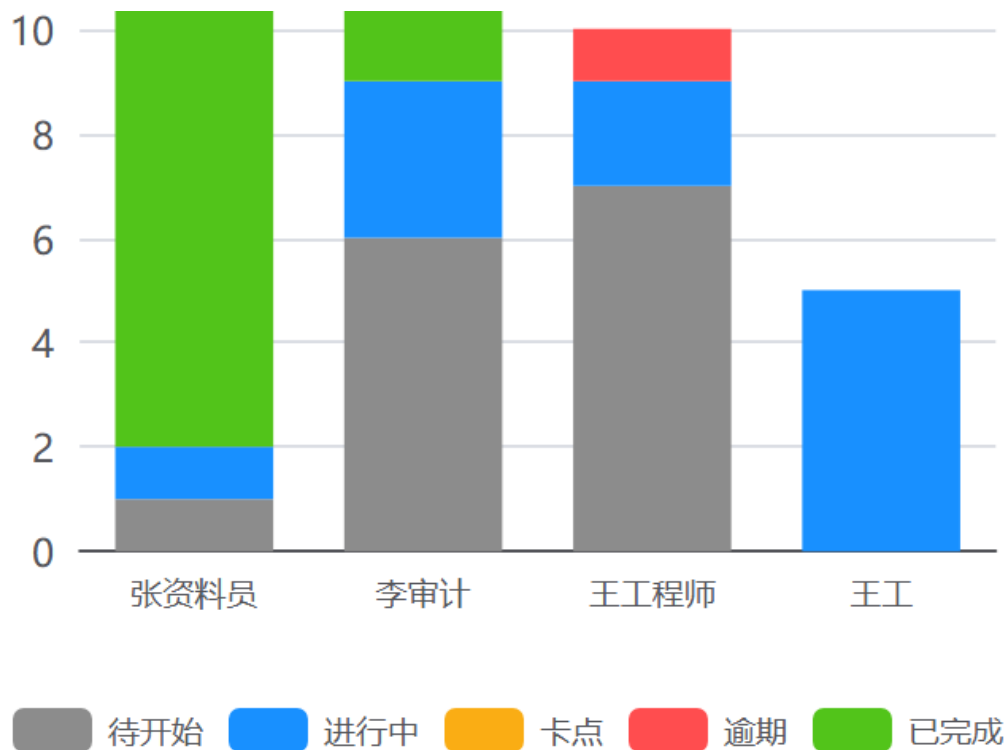
逾期



 团队工作量

12



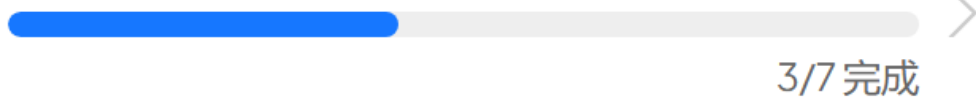


## 🏗️ 项目进度

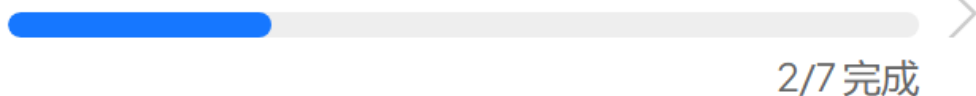
轨道交通5号线站点配套



市中心老旧小区改造三期



AA智慧产业园弱电项目



滨海湾地下综合管廊



### 项目看板

Trello 风格四列看板，支持拖拽流转任务状态。



## 轨道交通5号线站点配套



待开始

4

### 施工许可办理

下转: 开工进场

 逾期36天

王

### 材料进场验收

下转: 主体施工

 逾期33天

### 中期进度汇报

 逾期32天

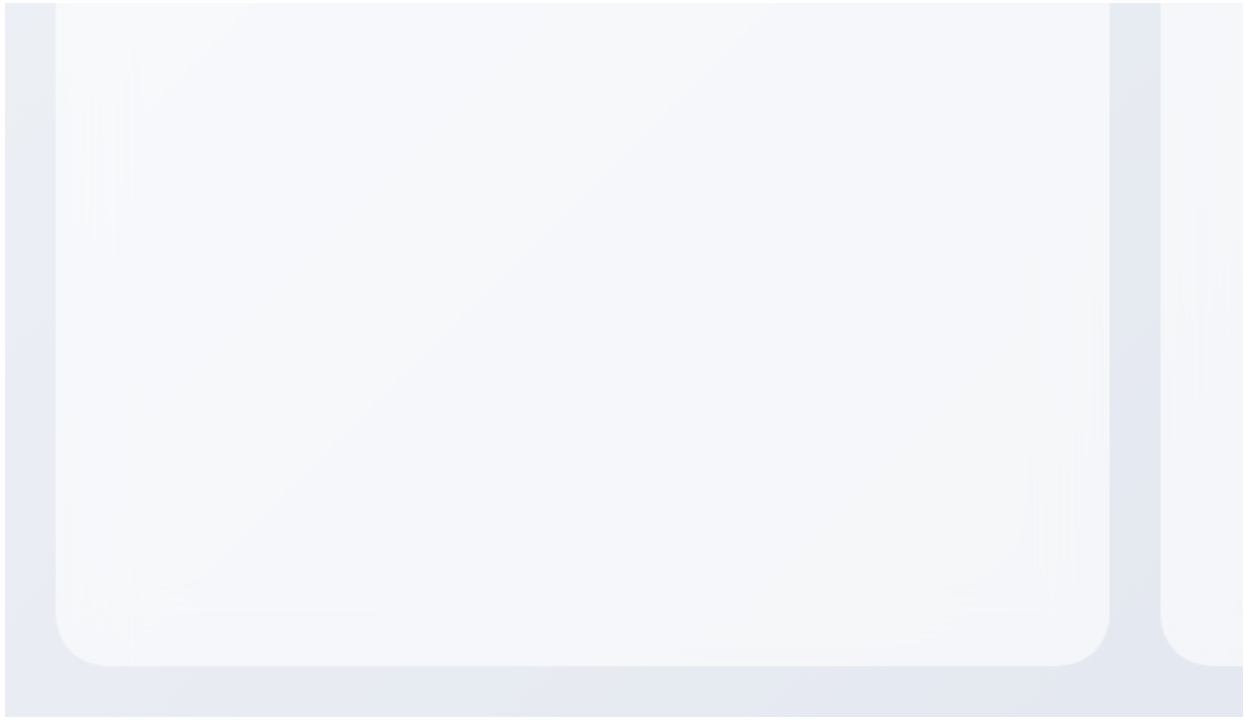
王

### 隐蔽工程验收

下转: 封顶

 逾期29天

李



### 项目时间轴

甘特图可视化，按人员分组，支持缩放。



# 轨道交通5号线站点配套

今天



横屏

| 执行人 | 12/21<br>Sun                                                                                                                  | 12/22<br>Mon | 12/23<br>Tue | 12/24<br>Wed                                                                                                          | 12/25<br>Thu |
|-----|-------------------------------------------------------------------------------------------------------------------------------|--------------|--------------|-----------------------------------------------------------------------------------------------------------------------|--------------|
|     | <div></div> <div>张资料员</div> <div>2任务</div>   |              |              | <div>方案设计</div> <div> 12/22</div>  |              |
|     | <div></div> <div>李审计</div> <div>2任务</div>    |              |              | <div>图纸审核</div> <div> 逾期</div>     |              |
|     |                                                                                                                               |              |              | <div>隐蔽工程验收</div> <div> 逾期</div> |              |
|     | <div></div> <div>王工程师</div> <div>2任务</div> |              |              | <div>施工许可办理</div> <div> 逾期</div> |              |
|     |                                                                                                                               |              |              | <div>中期进度汇报</div> <div> 逾期</div> |              |
|     | <div></div>                                |              |              | <div>材料进场验收</div> <div> 逾期</div> |              |

项目概览 & 关键节点

当前项目  
轨道交通5号线站点配套  
进行中



张资料员

任务: 6  
逾期  
阻塞



审核中心

交接提案列表，支持通过/驳回操作。



## 审核中心



暂无待审核的交接



## 系统设置

通知开关、密码修改、AI 配置、缓存清理。

## 系统设置

### 通用



消息通知



修改密码



清除缓存



### AI 设置



API Key

未配置 >



AI 模型

DeepSeek-V3 >

### 关于



帮助中心





关于版本

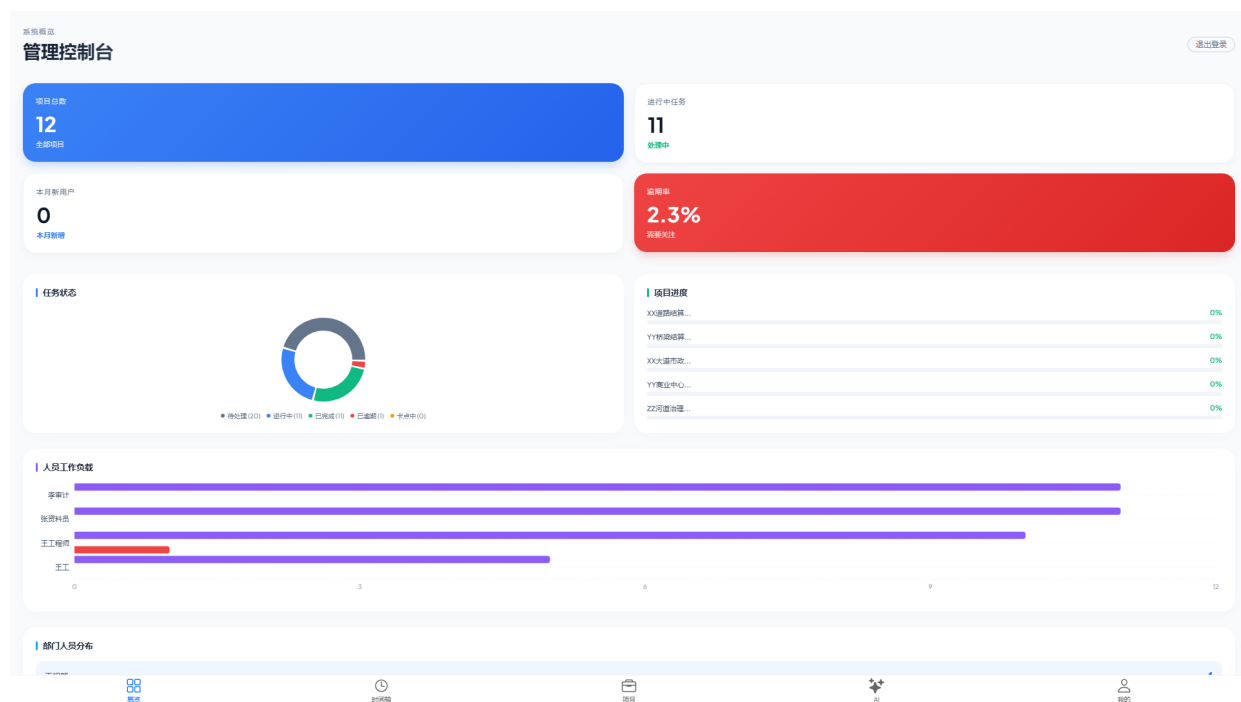
v2.1.0 >

Engineering Settlement System  
© 2026 All Rights Reserved

## 5.2.2 PC 端界面

### 管理控制台 (PC)

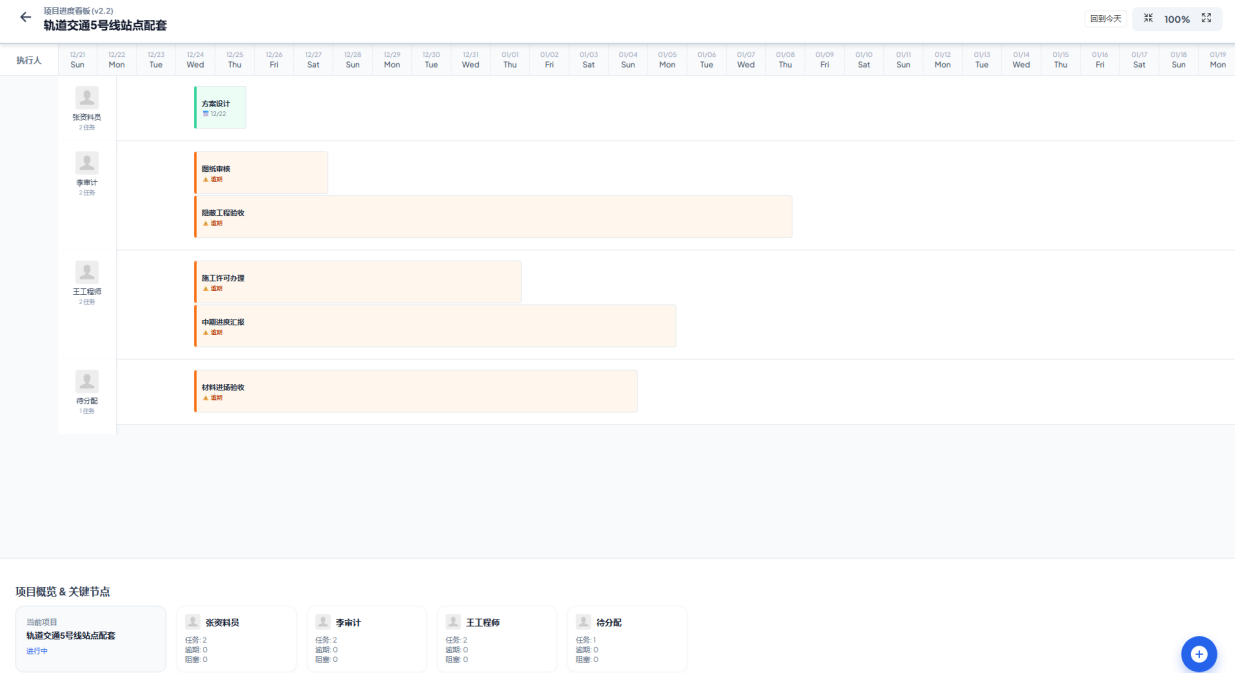
数据看板、ECharts 图表、用户管理、项目概览。



### 经理工作台 (PC)

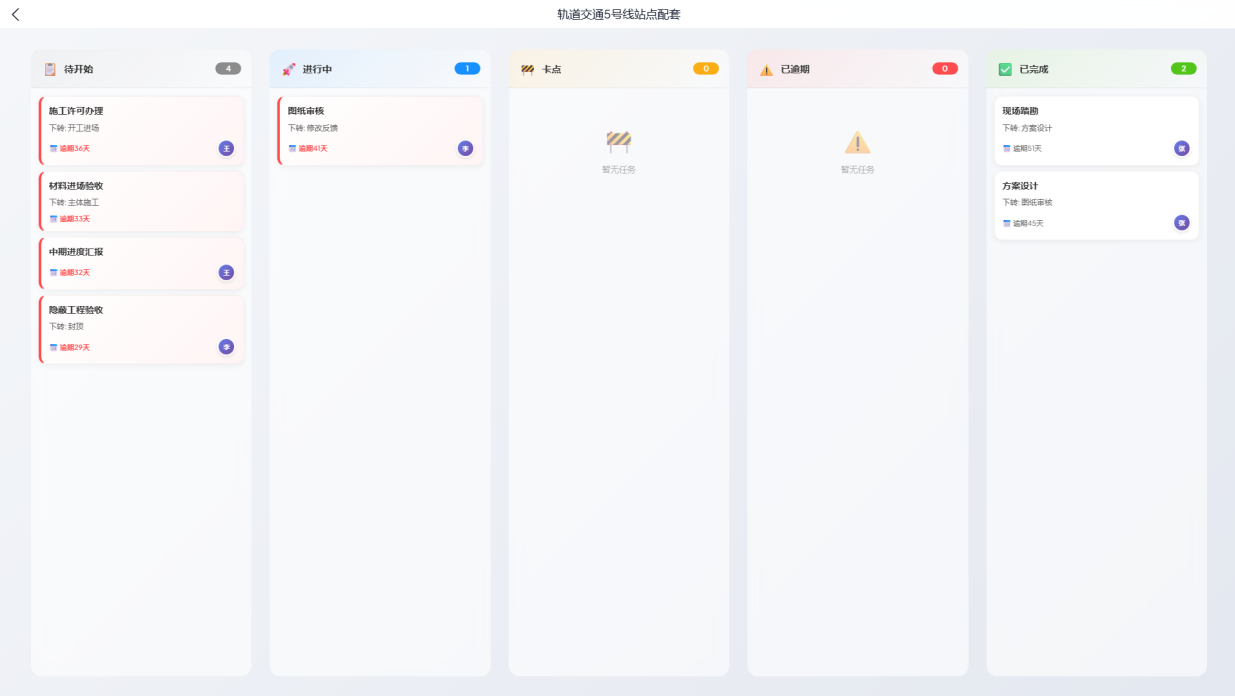
宽屏布局，关键指标一览无余。





项目看板 (PC)

五列看板，任务全览。



6. 部署指南

6.1 环境要求

|     |                          |
|-----|--------------------------|
| 组件  | 要求                       |
| 服务器 | Linux (推荐 CentOS/Ubuntu) |

| 组件      | 要求    |
|---------|-------|
| Node.js | 18.0+ |
| 内存      | 1GB+  |
| 磁盘      | 10GB+ |

## 6.2 后端部署 (PocketBase)

```
# 1. 上传 pocketbase 可执行文件
scp pocketbase root@your-server:/www/server/pocketbase/

# 2. 设置权限
chmod +x /www/server/pocketbase/pocketbase

# 3. 启动服务
cd /www/server/pocketbase
nohup ./pocketbase serve --http="0.0.0.0:8090" > pb.log 2>&1 &

# 4. 验证
curl http://localhost:8090/api/health
```

## 6.3 前端部署 (Nginx)

```
server {
    listen 80;
    server_name your-domain.com;

    root /www/wwwroot/app;
    index index.html;

    # SPA 路由支持
    location / {
        try_files $uri $uri/ /index.html;
    }

    # PocketBase 反代 (推荐)
    location /pb/ {
        proxy_pass http://127.0.0.1:8090/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # 静态资源缓存
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
```

## 6.4 数据库初始化

```
# 设置环境变量
export PB_URL="http://127.0.0.1:8090"
export PB_ADMIN_EMAIL="admin@example.com"
export PB_ADMIN_PASSWORD="your_password"

# 初始化数据库结构
cd v2/scripts
node database_rebuild.mjs
```

## 7. 使用指南

### 7.1 系统登录

登录步骤：

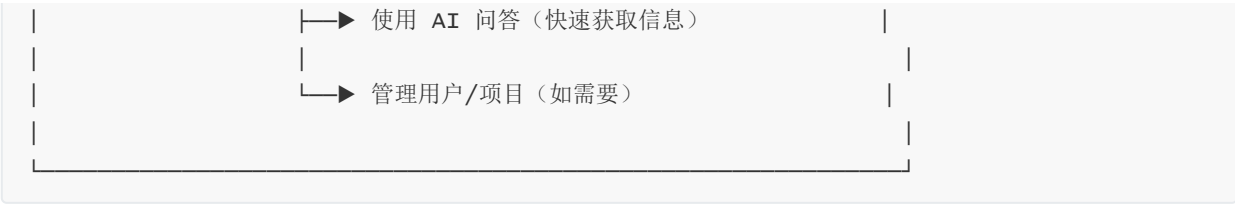
- 1. 打开系统地址（如 `http://localhost:5173`）
- 2. 输入用户名（或邮箱）和密码
- 3. 可选勾选"记住登录状态"
- 4. 点击"登录"按钮
- 5. 根据角色自动跳转到对应页面

角色默认跳转：

| 角色       | 跳转页面                           |
|----------|--------------------------------|
| admin    | <code>/admin</code> （管理控制台）    |
| manager  | <code>/admin</code> （管理控制台）    |
| employee | <code>/app/tasks</code> （工作进展） |

### 7.2 老板/管理员操作流程

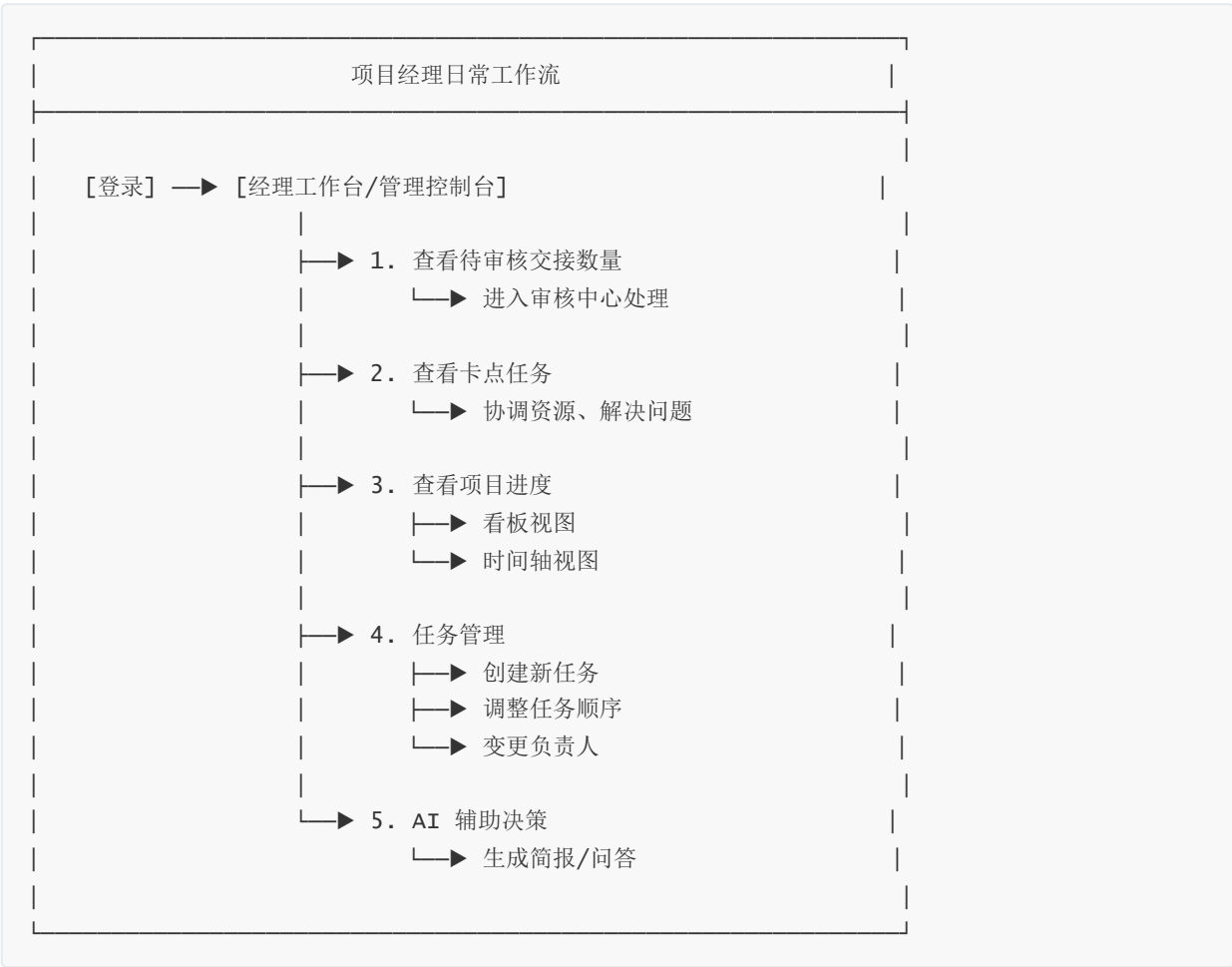




**关键操作：**

1. **查看 AI 简报：** 点击"AI 控制台"Tab → 点击"生成诊断报告"
2. **AI 问答：** 在 AI 控制台输入问题（如"当前最大风险是什么"）
3. **查看统计：** 数据概览展示关键指标和图表

### 7.3 项目经理操作流程

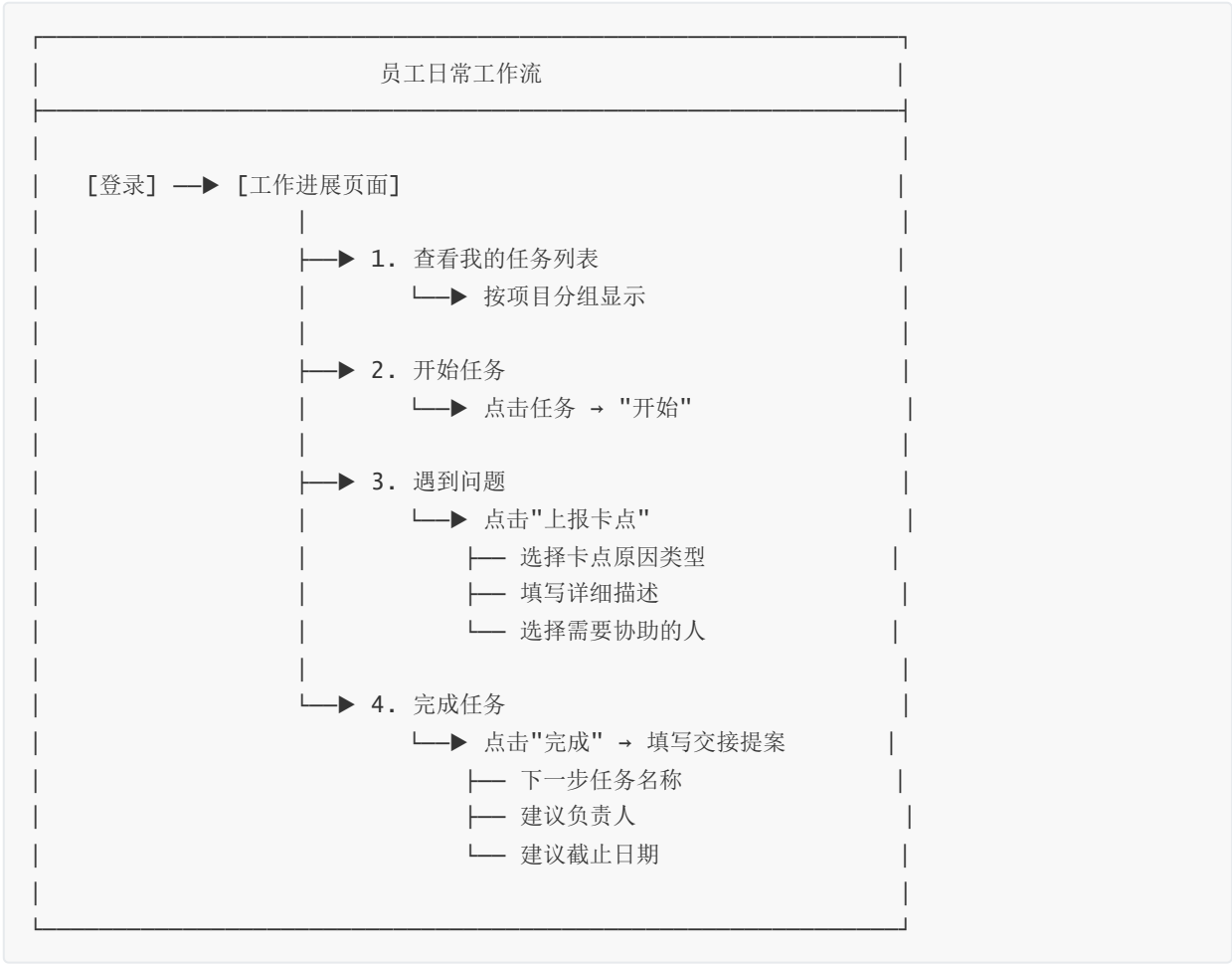


**审核交接流程：**

1. 进入"审核中心"页面
2. 查看待审核列表
3. 点击交接提案查看详情
4. 选择"通过"或"驳回"
  - 通过：系统自动创建新任务、通知新负责人
  - 驳回：填写驳回原因、通知提交人修改



## 7.4 员工操作流程



任务状态操作：

| 当前状态 | 可执行操作             |
|------|-------------------|
| 待开始  | 开始任务 → 进行中        |
| 进行中  | 上报卡点 → 卡点         |
| 进行中  | 完成任务 → 已完成（需提交交接） |
| 卡点   | 解决卡点 → 进行中        |
| 卡点   | 完成任务 → 已完成（需提交交接） |

## 7.5 AI 功能配置

配置方式：

方式 1：设置页面配置（推荐）

- 以 admin 或 manager 身份登录
- 进入"设置"页面
- 找到"AI 设置"区域

- 4. 输入 SiliconFlow API Key (sk-...)
- 5. 可选择 AI 模型
- 6. 点击保存

方式 2：浏览器控制台

```
// 设置 API Key
localStorage.setItem('sf_api_key', 'sk-your-api-key')

// 设置 AI 模型（可选）
localStorage.setItem('ai_model', 'deepseek-ai/DeepSeek-V3')
```

API Key 获取：

- 1. 访问 [SiliconFlow 官网](#)
- 2. 注册并登录账号
- 3. 在控制台创建 API Key
- 4. 复制 Key 到系统中配置

7.6 消息通知使用

通知类型：

| 类型   | 触发条件        | 接收人    |
|------|-------------|--------|
| 任务分配 | 新任务创建并指定负责人 | 负责人    |
| 卡点通知 | 员工上报卡点      | 项目经理   |
| 交接审核 | 员工提交交接提案    | 项目经理   |
| 审核结果 | 经理通过/驳回交接   | 提交人    |
| 任务逾期 | 任务超过截止日期    | 负责人+经理 |

通知操作：

- 点击通知可跳转到相关页面
- 可标记单条已读
- 可一键全部已读
- 可删除通知

8. API 接口

## 8.1 认证接口

```
POST /api/collections/users/auth-with-password
Content-Type: application/json
```

```
{
  "identity": "zhang_manager",
  "password": "12345678"
}

# 响应
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "record": {
    "id": "USER_ID",
    "username": "zhang_manager",
    "name": "张经理",
    "role": "manager",
    ...
  }
}
```

## 8.2 数据接口

### 项目列表

```
GET /api/collections/projects/records?sort=-created&expand=manager
```

```
# 响应
{
  "items": [
    {
      "id": "PROJECT_ID",
      "name": "凤凰山跨海大桥工程",
      "status": "active",
      "progress": 45,
      "expand": {
        "manager": { "id": "...", "name": "张经理" }
      }
    }
  ]
}
```

### 获取项目任务

```
GET /api/collections/tasks/records?
filter=project="PROJECT_ID"&expand=assignees&sort=sequence
```

```
# 响应
{
```

```

"items": [
  {
    "id": "TASK_ID",
    "stage_name": "地基验收",
    "status": "in_progress",
    "expand": {
      "assignees": [{ "id": "...", "name": "李工" }]
    }
  }
]
}

```

## 更新任务状态

```

PATCH /api/collections/tasks/records/TASK_ID
Content-Type: application/json
Authorization: Bearer TOKEN

{
  "status": "completed"
}

```

## 创建交接提案

```

POST /api/collections/handoffs/records
Content-Type: application/json
Authorization: Bearer TOKEN

{
  "project": "PROJECT_ID",
  "from_task": "TASK_ID",
  "proposed_title": "主体施工",
  "proposed_assignees": ["USER_ID"],
  "proposed_due_date": "2026-03-01",
  "status": "pending",
  "submitter": "USER_ID"
}

```

## 8.3 实时订阅

```

// 订阅通知
pb.collection('notifications').subscribe('*', (e) => {
  if (e.action === 'create') {
    console.log('新通知:', e.record)
    // 更新 UI 或显示提示
  }
})

// 订阅任务变更
pb.collection('tasks').subscribe('*', (e) => {

```

```

console.log('任务变更:', e.action, e.record)
// 刷新看板数据
queryClient.invalidateQueries(['tasks'])
})

```

## 8.4 前端 API Hooks

### 查询 Hooks (useQuery)

| Hook                                  | 说明       | 参数                              | 返回类型                        |
|---------------------------------------|----------|---------------------------------|-----------------------------|
| <code>useProjects()</code>            | 获取所有项目   | -                               | <code>Project[]</code>      |
| <code>useProject(id)</code>           | 获取单个项目   | <code>id: string</code>         | <code>Project</code>        |
| <code>useTasks(projectId?)</code>     | 获取任务列表   | <code>projectId?: string</code> | <code>Task[]</code>         |
| <code>useTask(id)</code>              | 获取单个任务   | <code>id: string</code>         | <code>Task</code>           |
| <code>useMyTasks(userId)</code>       | 获取我的任务   | <code>userId: string</code>     | <code>Task[]</code>         |
| <code>useUsers()</code>               | 获取用户列表   | -                               | <code>User[]</code>         |
| <code>useCurrentUser()</code>         | 获取当前用户   | -                               | <code>User   null</code>    |
| <code>useHandoffs(status?)</code>     | 获取交接列表   | <code>status?: string</code>    | <code>Handoff[]</code>      |
| <code>usePendingHandoffs()</code>     | 获取待审核交接  | -                               | <code>Handoff[]</code>      |
| <code>useNotifications(userId)</code> | 获取通知     | <code>userId: string</code>     | <code>Notification[]</code> |
| <code>useUnreadCount(userId)</code>   | 获取未读数    | <code>userId: string</code>     | <code>number</code>         |
| <code>useAuditLogs(taskId?)</code>    | 获取审计日志   | <code>taskId?: string</code>    | <code>AuditLog[]</code>     |
| <code>useComments(taskId)</code>      | 获取评论     | <code>taskId: string</code>     | <code>Comment[]</code>      |
| <code>useAISummaries(userId)</code>   | 获取 AI 简报 | <code>userId: string</code>     | <code>AISummary[]</code>    |

### 变更 Hooks (useMutation)

| Hook                             | 说明   | 参数                                  | 副作用       |
|----------------------------------|------|-------------------------------------|-----------|
| <code>useUpdateTask()</code>     | 更新任务 | <code>{ id, data }</code>           | 刷新任务缓存    |
| <code>useCreateTask()</code>     | 创建任务 | <code>Partial&lt;Task&gt;</code>    | 刷新任务列表    |
| <code>useDeleteTask()</code>     | 删除任务 | <code>id: string</code>             | 刷新任务列表    |
| <code>useCreateHandoff()</code>  | 创建交接 | <code>Partial&lt;Handoff&gt;</code> | 刷新交接列表    |
| <code>useApproveHandoff()</code> | 审核通过 | <code>{ id, note? }</code>          | 创建新任务、发通知 |

| Hook                                   | 说明   | 参数                                  | 副作用      |
|----------------------------------------|------|-------------------------------------|----------|
| <code>useRejectHandoff()</code>        | 驳回交接 | <code>{ id, note }</code>           | 发通知      |
| <code>useMarkTaskBlocked()</code>      | 标记卡点 | <code>{ id, blocker }</code>        | 更新状态、发通知 |
| <code>useCreateComment()</code>        | 创建评论 | <code>Partial&lt;Comment&gt;</code> | 刷新评论列表   |
| <code>useMarkNotificationRead()</code> | 标记已读 | <code>id: string</code>             | 更新未读数    |
| <code>useMarkAllRead()</code>          | 全部已读 | <code>userId: string</code>         | 更新通知列表   |
| <code>useUpdateUser()</code>           | 更新用户 | <code>{ id, data }</code>           | 刷新用户信息   |

## 使用示例

```
// 页面组件中使用 Hooks
import { useTasks, useUpdateTask, useCreateHandoff } from '@/lib/api'

function TaskList({ projectId }: { projectId: string }) {
  // 查询任务列表
  const { data: tasks, isLoading, error } = useTasks(projectId)

  // 更新任务状态
  const updateTask = useUpdateTask()

  // 创建交接提案
  const createHandoff = useCreateHandoff()

  const handleStatusChange = (taskId: string, newStatus: TaskStatus) => {
    updateTask.mutate({
      id: taskId,
      data: { status: newStatus }
    }, {
      onSuccess: () => Toast.show({ content: '状态已更新' }),
      onError: (err) => Toast.show({ content: '更新失败' })
    })
  }

  const handleCompleterWithHandoff = async (task: Task) => {
    // 先更新状态为完成
    await updateTask.mutateAsync({
      id: task.id,
      data: { status: 'completed' }
    })

    // 再创建交接提案
    await createHandoff.mutateAsync({
      project: task.project,
      from_task: task.id,
      proposed_title: '下一步任务',
      status: 'pending',
    })
  }
}
```

```
        submitter: pb.authStore.model?.id
      })
    }

    if (isLoading) return <Skeleton />
    if (error) return <ErrorState error={error} />

    return (
      <List>
        {tasks?.map(task => (
          <TaskCard
            key={task.id}
            task={task}
            onStatusChange={handleStatusChange}
            onComplete={handleCompleterewithHandoff}
          />
        ))}
      </List>
    )
  }
}
```

---

## 9. 安全设计

### 9.1 认证机制

- PocketBase 内置 JWT 认证
- Token 自动续期
- 密码加密存储

### 9.2 权限控制

- 基于角色的访问控制 (RBAC)
- 字段级权限 (员工只能改 status/blocker)
- API Rules 强制执行

### 9.3 数据安全

- SQLite 数据库加密 (可选)
- 定期备份 `pb_data/data.db`
- 敏感信息不写入代码仓库

### 9.4 AI Key 安全

- **当前**: 存储在 localStorage (本机)
  - **建议**: 迁移到服务端代理
-

## 10. 维护手册

### 10.1 日常运维

#### 10.1.1 服务状态检查

```
# 检查 PocketBase 进程
ps aux | grep pocketbase

# 检查端口占用
netstat -tlnp | grep 8090

# 健康检查
curl http://localhost:8090/api/health
# 预期返回: {"message":"API is healthy.","code":200,...}
```

#### 10.1.2 日志查看

```
# PocketBase 运行日志
tail -f /www/server/pocketbase/pb.log

# Nginx 访问日志
tail -f /www/wwwlogs/your-domain.com.log

# Nginx 错误日志
tail -f /www/wwwlogs/your-domain.com.error.log

# 实时监控最近 100 行
tail -n 100 -f /www/server/pocketbase/pb.log
```

#### 10.1.3 服务重启

```
# 停止 PocketBase
killall -f pocketbase

# 启动 PocketBase
cd /www/server/pocketbase
nohup ./pocketbase serve --http="0.0.0.0:8090" > pb.log 2>&1 &

# 重启 Nginx
nginx -t && nginx -s reload
```

### 10.2 数据备份与恢复



### 10.2.1 备份策略

| 备份类型 | 频率     | 保留时间  | 说明    |
|------|--------|-------|-------|
| 每日备份 | 每天凌晨   | 7 天   | 自动备份  |
| 周备份  | 每周日    | 4 周   | 自动备份  |
| 月备份  | 每月 1 日 | 12 个月 | 自动备份  |
| 手动备份 | 按需     | 永久    | 重大变更前 |

### 10.2.2 备份脚本

```
#!/bin/bash
# backup_pocketbase.sh

BACKUP_DIR="/backup/pocketbase"
DATE=$(date +%Y%m%d_%H%M%S)
PB_DATA="/www/server/pocketbase/pb_data"

# 创建备份目录
mkdir -p $BACKUP_DIR

# 备份数据库
cp $PB_DATA/data.db $BACKUP_DIR/data_$DATE.db

# 压缩备份
gzip $BACKUP_DIR/data_$DATE.db

# 清理 7 天前的备份
find $BACKUP_DIR -name "data_*.db.gz" -mtime +7 -delete

echo "Backup completed: data_$DATE.db.gz"
```

### 10.2.3 数据恢复

```
# 1. 停止 PocketBase
kill -f pocketbase

# 2. 备份当前数据（以防万一）
cp /www/server/pocketbase/pb_data/data.db /www/server/pocketbase/pb_data/data.db.bak

# 3. 恢复备份
gunzip /backup/pocketbase/data_20260204.db.gz
cp /backup/pocketbase/data_20260204.db /www/server/pocketbase/pb_data/data.db

# 4. 重启 PocketBase
cd /www/server/pocketbase
nohup ./pocketbase serve --http="0.0.0.0:8090" > pb.log 2>&1 &
```

## 10.3 常见问题排查

### 10.3.1 问题诊断表

| 症状     | 可能原因           | 排查步骤            | 解决方案         |
|--------|----------------|-----------------|--------------|
| 登录失败   | 用户名/密码错误       | 检查凭证            | 重置密码         |
| 登录失败   | PocketBase 未运行 | 检查进程            | 启动服务         |
| 页面白屏   | 前端资源加载失败       | 检查 Network      | 检查 Nginx     |
| 页面白屏   | SPA 路由问题       | 检查 Nginx 配置     | 添加 try_files |
| 看板拖拽失效 | 触摸事件未触发        | 检查长按时间          | 长按 250ms     |
| AI 无响应 | API Key 未配置    | 检查 localStorage | 配置 Key       |
| AI 无响应 | API Key 无效     | 测试 API          | 更换 Key       |
| AI 超时  | 网络问题/模型繁忙      | 检查网络            | 重试或换模型       |
| 通知不显示  | WebSocket 连接失败 | 检查控制台           | 刷新页面         |
| 数据不更新  | 缓存问题           | 清除缓存            | 设置页清除        |

### 10.3.2 详细排查指南

#### 问题：登录一直失败

```
# 1. 检查 PocketBase 是否运行
curl http://localhost:8090/api/health

# 2. 检查用户是否存在
curl "http://localhost:8090/api/collections/users/records?
filter=username='zhang_manager'"

# 3. 尝试直接调用认证 API
curl -X POST http://localhost:8090/api/collections/users/auth-with-password \
-H "Content-Type: application/json" \
-d '{"identity":"zhang_manager","password":"12345678"}'
```

#### 问题：AI 功能不工作

```
// 浏览器控制台执行
// 1. 检查 API Key 是否配置
console.log(localStorage.getItem('sf_api_key'))

// 2. 测试 API 连通性
fetch('https://api.siliconflow.cn/v1/chat/completions', {
  method: 'POST',
  headers: {
```

```
'Content-Type': 'application/json',
'Authorization': `Bearer ${localStorage.getItem('sf_api_key')}`
},
body: JSON.stringify({
  model: 'deepseek-ai/DeepSeek-V3',
  messages: [{ role: 'user', content: 'Hello' }]
})
}).then(r => r.json()).then(console.log).catch(console.error)
```

## 10.4 脚本工具说明

### 10.4.1 脚本列表

| 脚本                      | 功能           | 使用场景    | 危险等级      |
|-------------------------|--------------|---------|-----------|
| database_rebuild.mjs    | 重建数据库结构+演示数据 | 初始化/重置  | ⚠ 高（清空数据） |
| diagnose_and_fix_ai.mjs | 诊断修复 AI 相关集合 | AI 功能异常 | 🟢 低       |
| check_ai_status.mjs     | 检查 AI 功能状态   | 日常检查    | 🟢 低       |
| fix_ai_and_avatars.mjs  | 修复 AI + 添加头像 | 功能修复    | 🟡 中       |
| migrate_status.mjs      | 状态字段迁移       | 版本升级    | 🟡 中       |
| add_blocked_tasks.mjs   | 添加卡点任务数据     | 测试数据    | 🟢 低       |
| batch_assign_tasks.mjs  | 批量分配任务       | 数据处理    | 🟡 中       |
| test_all.mjs            | 全面系统测试       | 部署前检查   | 🟢 低       |
| screenshot_pages.py     | 自动截图         | 文档更新    | 🟢 低       |

### 10.4.2 脚本使用示例

```
# 设置环境变量
export PB_URL="http://127.0.0.1:8090"
export PB_ADMIN_EMAIL="admin@example.com"
export PB_ADMIN_PASSWORD="your_password"

# 运行数据库重建（⚠ 会清空数据）
cd v2/scripts
node database_rebuild.mjs

# 运行 AI 诊断
node check_ai_status.mjs

# 运行全面测试
node test_all.mjs

# 运行截图脚本
python screenshot_pages.py
```

## 10.5 性能优化建议

| 优化项     | 建议              | 优先级 |
|---------|-----------------|-----|
| 数据库索引   | 为常用查询字段添加索引     | 高   |
| 静态资源缓存  | Nginx 配置 30 天缓存 | 高   |
| Gzip 压缩 | 启用 Nginx Gzip   | 中   |
| CDN 加速  | 静态资源上 CDN       | 中   |
| 数据库清理   | 定期清理过期通知        | 低   |

## 10.6 监控告警建议

| 监控项     | 告警阈值       | 告警方式  |
|---------|------------|-------|
| 服务可用性   | 连续 3 次检测失败 | 短信/邮件 |
| CPU 使用率 | > 80%      | 邮件    |
| 内存使用率   | > 85%      | 邮件    |
| 磁盘使用率   | > 90%      | 邮件    |
| 响应时间    | > 3s       | 邮件    |

# 11. 验收标准

## 11.1 功能验收

| 验收项   | 验收标准                                    | 状态 |
|-------|-----------------------------------------|----|
| 流程闭环  | 员工完成任务 → 提交交接 → 经理审核通过 → 自动创建新任务，全链路无报错 | ✓  |
| 卡点追踪  | 卡点任务在看板高亮显示，仪表盘统计准确                     | ✓  |
| 动态调整  | 经理插入/删除任务后，成员刷新可见                       | ✓  |
| AI 生成 | 生成简报 30 秒内返回，正确渲染 Markdown              | ✓  |
| 多端适配  | Android APK 触控友好，Web PC 鼠标交互顺滑          | ✓  |

## 11.2 性能验收

| 指标     | 目标       | 实测         |
|--------|----------|------------|
| 页面加载   | < 3s     | ~1.5s      |
| 看板任务数  | > 500 可用 | 测试 100+ 流畅 |
| API 响应 | < 500ms  | ~200ms     |

## 11.3 安全验收

| 检查项       | 状态                |
|-----------|-------------------|
| JWT 认证    | ✓                 |
| RBAC 权限控制 | ✓                 |
| 敏感信息不入库   | ✓                 |
| SQL 注入防护  | ✓ (PocketBase 内置) |

## 12. 待优化项

### 12.1 功能优化

| 功能   | 当前状态 | 优化建议        |
|------|------|-------------|
| 消息推送 | 站内通知 | 集成 FCM/APNs |
| 逾期检测 | 手动   | 后端定时任务      |
| 虚拟滚动 | 未实现  | 大数据量场景      |

### 12.2 安全优化

| 项目     | 当前方案         | 优化建议  |
|--------|--------------|-------|
| AI Key | localStorage | 服务端代理 |
| HTTPS  | 可选           | 生产必须  |

## 附录

## A. 测试账号

说明：以下账号通过 `v2/scripts/database_rebuild.mjs` 脚本生成

### A.1 管理员账号

| 用户名         | 姓名 | 角色    | 密码       | 说明         |
|-------------|----|-------|----------|------------|
| admin_zhang | 张总 | admin | 12345678 | 系统管理员，全部权限 |
| admin_boss  | 老板 | admin | 12345678 | 决策层，查看简报   |

### A.2 经理账号

| 用户名           | 姓名   | 角色      | 部门  | 密码       |
|---------------|------|---------|-----|----------|
| zhang_manager | 张经理  | manager | 工程部 | 12345678 |
| wang_manager  | 王经理  | manager | 资料部 | 12345678 |
| wang_eng      | 王工程师 | manager | 工程部 | 12345678 |

### A.3 员工账号

| 用户名         | 姓名   | 角色       | 部门  | 密码       |
|-------------|------|----------|-----|----------|
| zhang_doc   | 张资料员 | employee | 资料部 | 12345678 |
| li_audit    | 李审计  | employee | 审计部 | 12345678 |
| chen_doc    | 陈资料员 | employee | 资料部 | 12345678 |
| liu_finance | 刘财务  | employee | 财务部 | 12345678 |
| zhao_safety | 赵安全员 | employee | 安全部 | 12345678 |

⚠ 注意：生产环境请修改所有测试账号密码！

## B. 环境变量

### B.1 后端环境变量

| 变量                             | 说明    | 默认值 | 必填    |
|--------------------------------|-------|-----|-------|
| <code>PB_ADMIN_EMAIL</code>    | 管理员邮箱 | -   | 首次启动时 |
| <code>PB_ADMIN_PASSWORD</code> | 管理员密码 | -   | 首次启动时 |

B.2 前端环境变量

| 变量          | 说明            | 默认值  | 必填 |
|-------------|---------------|------|----|
| VITE_PB_URL | PocketBase 地址 | 自动推导 | ✗  |

B.3 脚本环境变量

| 变量                | 说明            | 示例                    | 必填 |
|-------------------|---------------|-----------------------|----|
| PB_URL            | PocketBase 地址 | http://127.0.0.1:8090 | ✓  |
| PB_ADMIN_EMAIL    | 管理员邮箱         | admin@example.com     | ✓  |
| PB_ADMIN_PASSWORD | 管理员密码         | your_password         | ✓  |

B.4 AI 配置 (localStorage)

| Key        | 说明                  | 示例                      |
|------------|---------------------|-------------------------|
| sf_api_key | SiliconFlow API Key | sk-xxx...               |
| ai_model   | AI 模型名称             | deepseek-ai/DeepSeek-V3 |

B.5 PocketBase URL 解析优先级

前端连接 PocketBase 的 URL 按以下优先级解析：

1. 环境变量：VITE\_PB\_URL

↓（未设置）

2. localStorage: pb\_url

↓（未设置）

3. 运行时推导：根据当前页面 URL 推导

- localhost:5173 → localhost:8090

- your-domain.com → your-domain.com:8090

↓（推导失败）

4. 默认兜底：http://127.0.0.1:8090

C. 文档索引

C.1 产品文档

| 文档       | 路径                                      | 说明       |
|----------|-----------------------------------------|----------|
| PRD 产品需求 | v2/frontend/docs/PRD_v2.0_Enterprise.md | 完整产品需求文档 |
| 交付文档     | v2/docs/交付文档_完整版.md                     | 本文档      |

## C.2 技术文档

| 文档    | 路径                                 | 说明         |
|-------|------------------------------------|------------|
| 技术设计  | v2/frontend/docs/TECH_SPEC_v2.0.md | 技术规格说明     |
| 代码架构  | v2/docs/代码架构文档.md                  | 前端代码架构     |
| 数据库设计 | v2/docs/数据库设计_PocketBase版.md       | 数据库 Schema |
| 项目结构  | v2/docs/项目结构说明.md                  | 目录结构说明     |

## C.3 部署文档

| 文档     | 路径                   | 说明             |
|--------|----------------------|----------------|
| 快速启动   | v2/docs/快速启动_命令行版.md | 本地启动速查         |
| 宝塔部署   | v2/docs/宝塔部署操作手册.md  | 服务器部署指南        |
| App 打包 | v2/docs/App打包指南.md   | Android APK 打包 |

## C.4 截图目录

| 目录   | 路径                            | 说明          |
|------|-------------------------------|-------------|
| 界面截图 | v2/docs/screenshots/          | 自动生成的 UI 截图 |
| 截图索引 | v2/docs/screenshots/README.md | 截图列表        |

## D. 第三方服务

| 服务          | 用途      | 官网                                                            |
|-------------|---------|---------------------------------------------------------------|
| PocketBase  | BaaS 后端 | <a href="https://pocketbase.io/">https://pocketbase.io/</a>   |
| SiliconFlow | AI 服务   | <a href="https://siliconflow.cn/">https://siliconflow.cn/</a> |
| DeepSeek    | AI 模型   | <a href="https://deepseek.com/">https://deepseek.com/</a>     |

## E. 技术栈版本

| 技术         | 版本   | 说明    |
|------------|------|-------|
| React      | 19.x | UI 框架 |
| TypeScript | 5.9+ | 类型系统  |
| Vite       | 6.0+ | 构建工具  |



| 技术                | 版本    | 说明     |
|-------------------|-------|--------|
| Ant Design Mobile | 5.42+ | UI 组件库 |
| TanStack Query    | 5.x   | 数据管理   |
| Zustand           | 5.x   | 状态管理   |
| Framer Motion     | 12.x  | 动画库    |
| ECharts           | 5.x   | 图表库    |
| @dnd-kit          | 6.x   | 拖拽库    |
| PocketBase        | 0.26+ | 后端服务   |
| Capacitor         | 5.x   | 移动打包   |
| Node.js           | 18+   | 运行时    |
| Python            | 3.10+ | 脚本运行时  |
| Playwright        | 1.55+ | 自动化测试  |

## F. 快速命令速查

```
# ===== 本地开发 =====
# 启动后端
cd v2/backend && ./pocketbase serve --http="0.0.0.0:8090"

# 启动前端
cd v2/frontend && npm run dev

# ===== 数据库 =====
# 重建数据库（⚠️ 会清空数据）
cd v2/scripts && node database_rebuild.mjs

# 检查 AI 状态
cd v2/scripts && node check_ai_status.mjs

# ===== 构建部署 =====
# 构建前端
cd v2/frontend && npm run build

# 同步 Capacitor
cd v2/frontend && npx cap sync android

# 打包 APK
cd v2/frontend/android && ./gradlew assembleDebug

# ===== 文档 =====
# 生成截图
```

```
cd v2/scripts && python screenshot_pages.py
```

## 版本历史

| 版本     | 日期         | 变更说明                 |
|--------|------------|----------------------|
| v2.0   | 2026-02-04 | 初始交付版本               |
| v2.0.1 | 2026-02-02 | AI 功能修复、UI 优化        |
| v1.x   | -          | 已废弃 (Django/Flutter) |

## 联系方式

如有问题或需要技术支持，请参考：

- 📖 项目文档: [v2/docs/](#)
- 🐛 问题追踪: [GitHub Issues](#)
- 💬 技术支持: 开发团队

工程项目协同管理系统 v2.0

构建高效的工程协作平台

文档版本: v2.0  
生成时间: 2026-02-04